



Beyond Locations: A Motion Range-Aware Similarity Join

Ke Li

University of Electronic Science and
Technology of China
Chengdu, China
like_like@std.uestc.edu.cn

Lisi Chen

University of Electronic Science and
Technology of China
Chengdu, China
lchen012@e.ntu.edu.sg

Shuo Shang*

University of Electronic Science and
Technology of China
Chengdu, China
jedi.shang@gmail.com

Christian S. Jensen

Aalborg University
Aalborg, Denmark
csj@cs.aau.dk

Panos Kalnis

King Abdullah University of Science
and Technology
Thuwal, Saudi Arabia
panos.kalnis@kaust.edu.sa

Abstract

With the proliferation of GPS-enabled devices such as smartphones, the querying of moving objects has attracted substantial attention, with studies covering joins, range and k NN queries, similarity queries, etc. Challenges arise due to variable sampling frequencies, potential inaccuracies in location samples, and the unavailability of locations between samples. Existing similarity joins often rely on discrete location samples, which fail to capture movement uncertainty and may miss meaningful interactions. To address this limitation, we propose Intersection Similarity Join (IS-Join), a novel approach that identifies object pairs based on the overlap of their motion ranges rather than location-based proximity. We define motion ranges as the spatial regions an object may traverse within a given time period, and introduce an intersection similarity measure that quantifies their overlap. To efficiently process IS-Join queries, we develop a Hybrid Ball-tree indexing structure with a repartitioning strategy, enabling scalable candidate filtering. Additionally, we introduce pre-checking and pruning techniques to further reduce computational overhead. Extensive experiments on two real-world trajectory datasets demonstrate that IS-Join significantly outperforms well-designed baselines, achieving up to a 3x reduction in runtime. Our work opens new opportunities for applications such as urban mobility analysis, traffic monitoring, wildlife tracking, and contact tracing.

CCS Concepts

• Information systems → Location based services.

Keywords

similarity join, location-based services, spatio-temporal data mining, moving object indexing

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1454-2/2025/08
<https://doi.org/10.1145/3711896.3736855>

ACM Reference Format:

Ke Li, Lisi Chen, Shuo Shang, Christian S. Jensen, and Panos Kalnis. 2025. Beyond Locations: A Motion Range-Aware Similarity Join. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3711896.3736855>

1 Introduction

The matching of similar pairs of moving objects, called similarity join, is a fundamental functionality in data management. Existing proposals in this area [1, 11, 31, 32, 38] typically model moving objects as sequences of timestamped location samples and measure similarity based on spatial proximity between these samples. However, real-world objects are often sampled at relatively low frequencies due to privacy concerns, commercial considerations, missing values, and expensive deployment costs [16], making location information between consecutive observations unavailable. To introduce more flexibility, previous studies represent an object by a minimum bounding rectangle and model its movement by a velocity-based representation [27, 29, 33, 41, 42]. Another approach [2] represents trajectories as symbolic sequences using piecewise aggregate approximation [39], mapping trajectory segments to symbols from a predefined alphabet. Despite these advancements, existing representations fail to capture movement between samples, particularly for sparsely sampled trajectories. As a result, qualified result pairs may be incorrectly missed. How to effectively model the movement intersections of moving objects and how to efficiently find all intersections given massive volumes of moving objects remains an open challenge.

In this light, we introduce the Intersection Similarity Join (IS-Join) problem for moving objects, incorporating motion ranges to account for possible locations visited between observed samples. Specifically, we define an object's motion range as the region it can traverse at a time point or within a given time period. The intersection similarity between two objects is determined by the overlap of features within their motion ranges. Given a collection of objects in table `MovingObject`, and a similarity threshold θ , we define the IS-Join in a SQL-style format as follows ¹.

¹For ease of representation, we define the IS-Join in a self-join style. Note that our proposal supports non-self join as well.

```

SELECT o.id, o'.id
FROM MovingObject AS o, MovingObject AS o'
WHERE o.id < o'.id AND IS(o, o', I) ≥ θ

```

Each object o is characterized by an id attribute, two location coordinate attributes, and two timestamp attributes that denote the time when these two locations are visited. The condition $o.id < o'.id$ prevents duplicate comparisons. The function $IS(o, o', I)$ measures the intersection similarity between two objects o and o' based on the overlap of features within two motion ranges during a time interval I . Parameter θ is a pre-defined similarity threshold. The IS-Join problem targets a variety of applications, such as disease control, trajectory data cleaning, ridesharing recommendation, and traffic congestion prediction [18, 36]. Let us consider the following potential application scenarios.

Scenario 1 - Urban Traffic Monitoring. The feature set may include road segments, intersections, and traffic signals. Intersection similarity captures the likelihood of two vehicles experiencing similar conditions, aiding in congestion prediction and traffic optimization. For point-based features such as POIs and road intersections, we assign a unique ID to each point. To apply a similar approach to road segments, we divide roads into equal-length sub-segments, each with a unique ID.

Scenario 2 - Wildlife Migration. The defined motion ranges help identify shared migration corridors by considering features such as habitats, water sources, and travel routes. In addition, motion ranges provide a balance between energy-efficient data collection and accurate interaction detection [13].

Scenario 3 - Contact Tracing. The defined motion ranges allow for an estimation of potential contact events, enhancing the effectiveness of epidemic monitoring systems. The intersection similarity can assist users in assessing their infection risk, which is valuable in mitigating infection during the Covid pandemic [19].

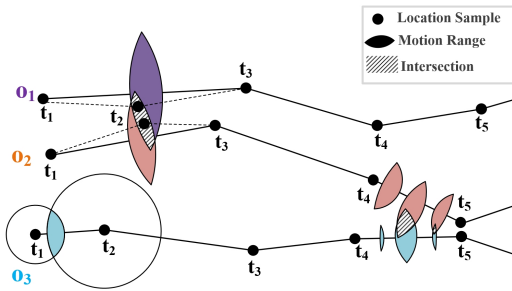


Figure 1: Example of the IS-Join problem

Example. Consider three moving objects o_1, o_2 , and o_3 in Figure 1, each with location samples from timestamps t_1 to t_5 . For simplicity, assume these location samples are time-aligned. The location samples of o_1 and o_2 at t_2 are either missing or too noisy to be useful. If we perform a similarity join by merely checking if the distances between location samples fall below a given threshold, we might incorrectly conclude that (o_1, o_2) does not qualify as a result pair during the time interval $[t_1, t_3]$, even if the distance was below the threshold at t_2 . We can avoid such false dismissals by considering the motion ranges of the objects. Consider locations of o_3 at t_1 and t_2 , the motion range of o_3 at time point $t \in [t_1, t_2]$

is a lens to be detailed in Section 2. The motion ranges of o_1 and o_2 at time point $t \in (t_1, t_3)$ are illustrated with purple and brown lenses, respectively. It is observed that the motion ranges of o_1 and o_2 overlap in the shadowed region, which indicates they may have potential location intersection. Next, we focus on the time interval $[t_4, t_5]$. A sequence of motion ranges of o_2 and o_3 at some discrete time points within $[t_4, t_5]$ is illustrated. The intersection similarity of interval $[t_4, t_5]$ is the averaged intersection similarity of the discrete time points in $[t_4, t_5]$, as will be explained later in the problem definition. The IS-Join problem, when provided with a time interval and a similarity threshold, identifies and returns all pairs of objects whose intersection similarities are equal to or greater than the specified threshold within the given time interval.

To the best of our knowledge, this is the first study on similarity joins over moving objects that explicitly incorporates motion ranges. For IS-Join to be practical, it must be efficient and scalable to massive datasets. However, solving the IS-Join problem presents two key challenges. First, computing exact intersection similarity over a time period is impractical, as it requires handling an infinite number of time points. Second, existing join solutions were designed for different query types and do not directly support IS-Join. Our experiments show that simply adapting these methods results in severe inefficiencies even with small datasets. To address this, we propose a time-interval motion range guided solution. Specifically, we develop an HBall-tree-based join algorithm that integrates effective repartitioning and pruning techniques, significantly reducing the average runtime by a factor of 3 compared to well-designed baseline solutions.

The main contributions of the paper are summarized as follows.

- We introduce a novel problem of Intersection Similarity Join (IS-Join) and investigate existing techniques that can be adapted to solve it. Unlike previous methods that identify similar pairs based on proximity at discrete location samples, our work is the first to detect pairs whose motion range-aware similarity surpasses a specified threshold.
- We propose a time-interval motion range guided solution to efficiently answer the IS-Join problem. To achieve this, we develop a Hybrid Ball-tree (HBall-tree) and an associated pruning enhancement technique. The HBall-tree efficiently identifies candidate pairs, while the pruning enhancement technique eliminates unqualified pairs early in the process, significantly improving query performance.
- We report on extensive experiments on two real-world datasets, offering insight into the proposed solutions and offering evidence that our approach significantly outperforms well-designed baseline methods.

2 Preliminaries and Problem Formulation

In this section, we define the motion range of a moving object and the intersection similarity between moving objects. We then introduce the concept of relaxed intersection similarity and formally define the intersection similarity join problem. The key notations used throughout this paper are summarized in Table 1.

Definition 1: (Timestamped Location Sample) A timestamped location sample of a moving object is represented as $s = ([x, y], t)$,

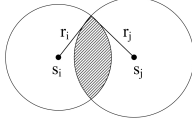


Figure 2: Illustration of time-point motion range

where $[x, y]$ denotes spatial coordinates and t is the timestamp when this location is visited.

Definition 2: (Time-Point Motion Range) Given location samples $s_i = ([x_i, y_i], t_i)$ and $s_j = ([x_j, y_j], t_j)$ of a moving object o , along with the maximum speed v_{max} , the motion range of object o at time t (where $t \in [t_i, t_j]$) is denoted by $MR(o, t)$ and calculated by Equation 1. The function $dist(\cdot)$ denotes the distance metric between two locations. This is a lens shape as illustrated in Figure 2, with radii r_i and r_j given by $v_{max} \cdot (t - t_i)$ and $v_{max} \cdot (t_j - t)$.

$$MR(o, t) = \{(x, y) | dist((x, y), (x_i, y_i)) \leq v_{max} \cdot (t - t_i) \wedge dist((x, y), (x_j, y_j)) \leq v_{max} \cdot (t_j - t)\} \quad (1)$$

The time-point motion range modeling is consistent with previous studies [28, 35, 43]. Our modeling uses the maximum speed to ensure that the estimated motion range fully covers the actual movement. In particular, the motion range at a sampled timestamp corresponds to the timestamped location sample itself. Objects with exceptionally long sampling intervals, such as several hours, are beyond the scope of our analysis.

Definition 3: (Motion Range Feature) Let $\mathcal{F}(o, t)$ represent the set of specific features within the motion range $MR(o, t)$. Examples include the area of the motion range, points of interest within the range, road segments traversed, and geo-spatial regions or environmental factors.

Definition 4: (Time-Point Intersection Similarity) The time-point intersection similarity between objects o and o' at time t , denoted by $IS(o, o', t)$, and defined by Equation 2, measures the overlap between their feature sets. Here, $|\mathcal{F}(\cdot)|$ represents the size of feature set within $MR(o, t)$, and $\cap$ denotes the intersection of the feature sets, which may include shared POIs, common road segments, or overlapping motion range areas, depending on the

application scenarios. The similarity score ranges from 0 to 1, where 1 indicates a perfect overlap of feature sets and 0 signifies no overlap.

$$IS(o, o', t) = \frac{|\mathcal{F}(o, t) \cap \mathcal{F}(o', t)|}{|\mathcal{F}(o, t)|} \times \frac{|\mathcal{F}(o, t) \cap \mathcal{F}(o', t)|}{|\mathcal{F}(o', t)|} \quad (2)$$

The time-point intersection similarity is based on the following rationale: First, $MR(o, t)$ models the possible locations of object o can occupy at time t , while $\mathcal{F}(\cdot)$ represents the set of relevant features within this motion range. Taking the product means that both factors must be sufficiently large for the intersection similarity to be high. This ensures that the similarity score is only high when both conditions—motion range overlap and feature similarity are strong. Classic Jaccard, averaging, and minimum methods fail to balance motion range overlap and feature similarity effectively. Consequently, $IS(o, o', t)$ quantifies the likelihood of interaction between o and o' by assessing the overlap of their respective feature sets.

Definition 5: (Time-Interval Intersection Similarity) Given a time-interval I with a start time t_s and an end time t_e , we define the intersection similarity between objects o and o' over I as the average of the time-point intersection similarities, as given in Equation 3.

$$IS(o, o', I) = \frac{\int_{t_s}^{t_e} IS(o, o', t) dt}{t_e - t_s} \quad (3)$$

Calculating the time-interval similarity is non-trivial, as it requires evaluating the time-point intersection similarity over an infinite number of time points due to the continuous nature of the interval. To provide a practical approximation and ensure scalability for large-scale moving objects in real-world applications, we propose a relaxed definition without significant loss of generality.

Definition 6: (ϵ -Relaxed Time-Interval Intersection Similarity) Given a small constant ϵ , we establish an ϵ -sampling of a time interval $I = [t_s, t_e]$ by uniformly sampling a set of time points $T^\epsilon = \{t_1^\epsilon, \dots, t_m^\epsilon\}$, where $t_1^\epsilon = t_s$, $t_m^\epsilon = t_e$ and $t_{i+1}^\epsilon - t_i^\epsilon = \epsilon$. We then define the ϵ -relaxed time-interval intersection similarity, denoted as $IS(o, o', I, \epsilon)$, as the average of $IS(o, o', t)$ over all time points $t^\epsilon \in T^\epsilon$, which is computed by Equation 4.

$$IS(o, o', I, \epsilon) = \frac{\sum_{i=1}^m IS(o, o', t_i^\epsilon)}{m} \quad (4)$$

Fundamentally, calculating time-interval similarity requires multiple evaluations of time-point intersection similarities. In the following sections, unless stated otherwise, we use “intersection similarity” to refer to the ϵ -relaxed intersection similarity for simplicity. When ϵ approaches 0, $IS(o, o', I, \epsilon)$ converges to $IS(o, o', I)$.

Example. Figure 3 illustrates an example of calculating the time-interval intersection similarity. Consider $I = (t_4, t_5)$ with $\epsilon = |I|/4$, resulting in the sampled time set $T^\epsilon = \{t_4, t_4 + \epsilon, t_4 + 2\epsilon, t_4 + 3\epsilon, t_5\}$. Two moving objects, o_2 and o_3 , have their time-point motion ranges within T^ϵ . At time points t_4 and $t_4 + \epsilon$, no intersection occurs, yielding $IS(o_2, o_3, t_4) = 0$ and $IS(o_2, o_3, t_4 + \epsilon) = 0$. At t_5 , the location samples of o_2, o_3 are the same, resulting in $IS(o_2, o_3, t_5) = 1$. Thus, the ϵ -relaxed time-interval intersection similarity over I is computed as $(0+0+IS(o_2, o_3, t_4+2\epsilon)+IS(o_2, o_3, t_4+3\epsilon)+1)/5$. The values $IS(o_2, o_3, t_4 + 2\epsilon)$ and $IS(o_2, o_3, t_4 + 3\epsilon)$ are determined using Equation 2, which is specifically designed to handle scenario-dependent calculations.

Notation	Definition
o	A moving object
s	A timestamped location sample
$[x, y], t$	The coordinates / timestamp
I	A time interval
$MR(\cdot)$	A motion range of an object
$dist(\cdot)$	The distance metric
$\mathcal{F}(\cdot)$	The feature set of a motion range
$IS(\cdot)$	The intersection similarity metric
θ	The intersection similarity threshold
ϵ	The relaxed partition parameter
T^ϵ	The sampled timestamp set
$\mathcal{A}, \mathcal{B}$	Two sets of moving objects
$\mathcal{R}$	The IS-Join results

Table 1: The summary of notations

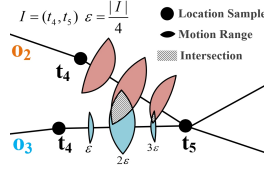


Figure 3: Illustration of time-Interval intersection similarity

We formally define the intersection similarity join problem in Definition 7.

Definition 7: (IS-Join) Given sets $\mathcal{A}$ and $\mathcal{B}$ of objects, a query time interval I with a relaxed partitioning parameter ϵ , and a similarity threshold θ , the Intersection Similarity Join (IS-Join) finds the set $\mathcal{P}$ of all objects from the two sets whose intersection similarity no smaller than θ , i.e., $\forall (o_i, o_j) \in (\mathcal{A} \times \mathcal{B}) \setminus \mathcal{P} \ (IS(o_i, o_j, I, \epsilon) < \theta)$.

3 Motion Range Guided Matching Approach

To efficiently answer IS-Joins, we propose a time-interval motion range guided matching approach. In the time-interval phase, we perform a pre-check to determine whether any intersection exists between the time-point motion ranges of two objects. Specifically, we define a time-interval motion range $MR(o, I)$ that encompasses all possible locations an object o may travel during the interval I . If $MR(o, I) \cap MR(o', I) = \emptyset$, it follows that $IS(o, o', I, \epsilon) = 0$, guiding us to prune the pair (o, o') . To efficiently determine whether $MR(o, I) \cap MR(o', I) = \emptyset$, we propose a novel Hybrid Ball-tree (HBall-tree) structure. We introduce a pruning enhancement method that utilizes upper bounds on intersection similarity to further eliminate unqualified object pairs, refining the overall process. In the refinement phase, the intersection of features within motion ranges are computed for measuring the intersection similarity. We first introduce the time-interval motion range guided pre-checking strategy, followed by the Hybrid Ball-tree based join (HBJ-Join) algorithm.

3.1 Time-Interval Pre-Checking

In this section, we introduce the concept of the time-interval motion range and the pre-checking strategy with provable theorems.

Definition 8: (Time-Interval Motion Range) Given two time-stamped location samples $s_i = ([x_i, y_i], t_i)$ and $s_j = ([x_j, y_j], t_j)$ of moving object o , and the maximum speed v_{max} between t_i and t_j , the motion range of object o during a time interval $I = [t_i, t_j]$, denoted by $MR(o, I)$, is defined as the spatial region within which o may travel during I .

Theorem 3.1 provides the method for computing $MR(o, I)$.

THEOREM 3.1. *The motion range $MR(o, I)$ of object o is the elliptical region represented by Equation 5. The center of the ellipse, denoted by (x_c, y_c) , is calculated using Equation 6. The rotation angle of this ellipse is denoted by α , computed by Equation 7, where a and b represent the semi-major and semi-minor axes, respectively.*

$$\left[\frac{(x - x_c) \cos \alpha + (y - y_c) \sin \alpha}{a} \right]^2 + \left[\frac{(x_c - x) \sin \alpha + (y - y_c) \cos \alpha}{b} \right]^2 = 1 \quad (5)$$

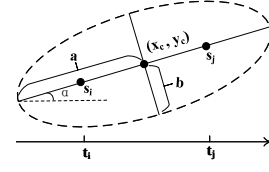


Figure 4: Example of time-interval motion range

$$x_c = \frac{x_i + x_j}{2}, y_c = \frac{y_i + y_j}{2}, a = \frac{v_{max} \cdot (t_j - t_i)}{2} \quad (6)$$

$$b = \frac{\sqrt{v_{max}^2 \cdot (t_j - t_i)^2 - (x_j - x_i)^2 - (y_j - y_i)^2}}{2}$$

$$\alpha = \arctan\left(\frac{y_j - y_i}{x_j - x_i}\right) \quad (7)$$

Proof: The maximum distance that o can travel during the time interval I is $(t_j - t_i) \cdot v_{max}$, applicable to both Euclidean and road network spaces. Given two location samples s_i and s_j , the motion range of object o at a specific time point t is defined by Equation 1. Notably, the sum of distances from any point on the ellipse to its two foci is a constant $2a$. If $v_{max} \cdot (t_j - t_i) = 2a$, the possible travel locations from t_i to t_j form an elliptical range with foci at (x_i, y_i) and (x_j, y_j) , as illustrated in Figure 4. Our modeling of time-interval motion range aligns with existing studies [35, 43]. $\square$

A pre-checking strategy is developed based on Theorem 3.2: If $MR(o, I) \cap MR(o', I) = \emptyset$, we can safely prune the pair (o, o') at an early stage, avoiding unnecessary computations for the intersection of feature sets (e.g., POIs, area of the motion range).

THEOREM 3.2. *Given two objects o and o' , if $MR(o, I) \cap MR(o', I) = \emptyset$, then $IS(o, o', I, \epsilon) = 0$.*

Proof: For objects o and o' , if their time-interval motion ranges during interval I do not intersect, no time-point motion ranges of o and o' within I can overlap. Since the feature sets $P(o, t)$ and $P(o', t)$ associated with their respective time-point motion ranges, it follows that there is no intersection between these feature sets throughout I . Consequently, the intersection similarity over I is 0. $\square$

To determine whether two time-interval motion ranges intersect, a straightforward approach is to compute their pairwise distance. Specifically, if the distance $dist(MR(o, I), MR(o', I))$ exceeds $MR(o, I).a + MR(o').a$, then the motion ranges do not intersect. Here, $dist(MR(o, I), MR(o', I))$ is the distance between the centers of $MR(o, I)$ and $MR(o', I)$, and a represents the semi-major axis of the motion range. The straightforward approach requires $O(\mathcal{A} \times \mathcal{B})$ time complexity for one-to-one comparisons, making it computationally expensive.

3.2 Intersection Pre-checking using HBall-tree

We introduce the Hybrid Ball-tree (HBall-tree), an improved index structure designed for efficient pre-checking, and demonstrate how it enhances the efficiency of the filter-and-refine process.

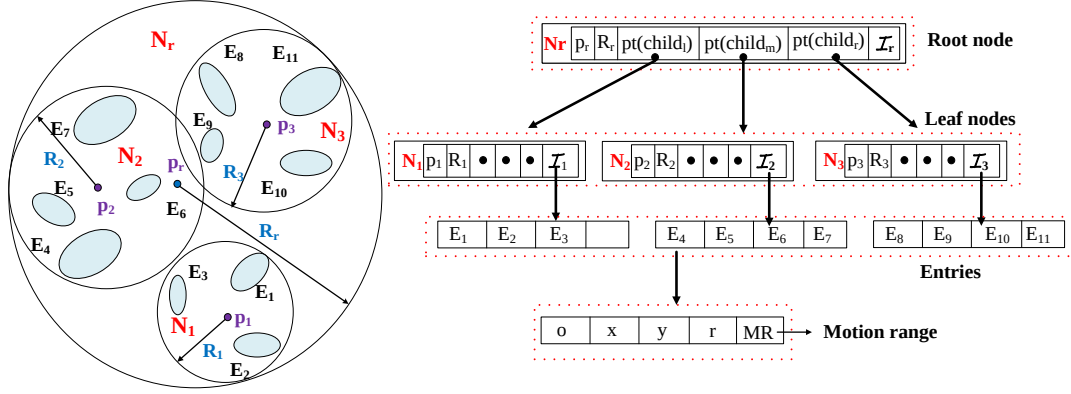


Figure 5: Illustration of the hybrid ball-tree

3.2.1 The HBall-tree. We use the Ball-tree [22, 23, 26] as the underlying index due to its superior performance with circle-like objects [8, 9, 43], as compared to other indexes such as the R-tree [14] and quadtree [30]. However, Ball-trees still face two key limitations when applied to IS-Join queries. First, they are primarily designed for nearest-neighbor searches at the point or circle level and do not directly support indexing time-interval motion ranges. Second, their partitioning process does not account for the distribution of data points, which may result in uneven partitions and a significantly unbalanced tree structure [12].

To address the first challenge, we approximate each time-interval motion range by a circle [43]. Specifically, the circle is centered at the center of the time-interval motion range, and has a radius equal to the semi-major axis. This approximation simplifies the representation of time-interval motion ranges and allows us to leverage existing spatial indexes designed for circular objects. The key inference here is that if two approximate circles do not intersect, their corresponding motion ranges also do not intersect. This inference underpins our filtering technique, facilitating the efficient identification of object pairs unlikely to intersect. To address the second challenge, we introduce the Hybrid Ball-tree (HBall-tree), a novel index structure that takes specific data distribution into account. Figure 5 provides an illustration of the HBall-tree. Each time-interval motion range is represented in the format of $E = \{o, x, y, r, MR(o, I)\}$, where (x, y) and r are the center and radius of the approximate circle, respectively. Notation o represents the object identifier. Unlike the original Ball-tree, which has exactly two children for each non-leaf node, the HBall-tree supports up to three children per parent node through a repartitioning technique. The structure of each node follows the format $N = \{p, R, pt(child_l), pt(child_m), pt(child_r), I\}$, where p is the center point of the time-interval motion ranges covered by node N . The notation R is a radius that can enclose all entries (i.e., circles) stored in N , and $pt()$ serves as a pointer to its child nodes. The notation I denotes the set of time-interval motion ranges within each node. The entry insertion policy, splitting strategies, and pivot selection in the HBall-tree follow established practices from existing studies [23]. Sub-partitioning of a node stops when the number of objects in its sub-partitions falls below a predefined threshold, which indicates the minimum number of objects required to create a new child node.

3.2.2 Repartition for HBall-tree. Intuitively, a more balanced tree structure generally results in fewer node accesses during searches. The high-level idea of the repartitioning method works as follows: For each original partition, we evaluate whether the number of points in the two sub-partitions differs by more than a re-partition threshold β ($\beta \in (0, 1)$). If the difference exceeds the threshold, we retain the sub-partition with fewer points and further partition the sub-partition with more points into two new sub-partitions. Let us consider a node N . Initially, $N.child_m = \emptyset$. repartitioning is triggered if the size of the larger sub-partition exceeds the smaller one by a ratio of at least β . For example, if $|N.child_l.I| > |N.child_r.I|$, and $\frac{|N.child_l.I| - |N.child_r.I|}{|N.child_r.I|} > \beta$, repartitioning is triggered. In theory, a smaller value of re-partition threshold means that initial subtrees are more likely to be re-partitioned, which increases the number of ternary nodes, resulting in a more balanced tree. However, the introduction of more ternary nodes may increase node accesses. Therefore, it is crucial to strike a balance between tree height and the number of ternary nodes.

For a non-leaf node, the initial partitioning is performed based on two farthest points selected within $N.I$. Specifically, the two farthest points during tree construction are selected as follows: (i) Random Seed Point: Choose an arbitrary point from $N.I$ as the initial seed. (ii) First Farthest Point: Find the point in the dataset that is farthest from the seed. (iii) Second Farthest Point: Find the point that is farthest from the first farthest point. Given the previously selected two farthest points within $N.I$, namely $N.child_l.p$ and $N.child_r.p$, we compute the middle point between $N.child_l.p$ and $N.child_r.p$. Next, we select the nearest center points within $N.I$ to the middle point, denoted as p_m . The sub-partition with the smaller size is considered stable, and we aim to re-partition the larger one. Specifically, if the original partitions have $|N.child_l.I| > |N.child_r.I|$, then we use p_m and $N.child_l.p$ to re-partition motion ranges within $N.child_l.I$ into two new sub-partitions: one remains in $N.child_l.I$, and the other is assigned to $N.child_m.I$. If the original partitions have $|N.child_l.I| < |N.child_r.I|$, then we use p_m and $N.child_r.p$ to re-partition motion ranges within $N.child_r.I$ into two new sub-partitions, one stays in $N.child_r.I$, and the other is assigned to $N.child_m.I$. As a result, the repartitioning divides the existing two sub-partitions into three sub-partitions, with the new sub-partition

indexed by $N.pt(child_m)$. Our re-partition technique offers two main benefits: (1) Reduction in tree height and partitions: By improving the balance, the height of the tree is reduced, and the total number of partitions is minimized. (2) Enhanced search efficiency: A more balanced tree structure improves search efficiency by decreasing the number of tree node accesses.

3.3 HBall-tree based Join

We introduce a HBall-tree based Join algorithm. To enhance efficiency, we implement a pruning enhancement method that leverages upper bounds on intersection similarity to further eliminate unqualified object pairs, refining the overall process.

3.3.1 Pruning Enhancement Strategy. It is non-trivial to compute the intersection of two feature sets $\mathcal{F}(o, t)$ and $\mathcal{F}(o', t)$ due to the irregular shape of the intersection of motion ranges. Given a time point t , if $MR(o, t) \cap MR(o', t) \neq \emptyset$, the value of $IS(o, o', t)$ reaches its maximum value when one object's motion range is entirely covered by the other. In view of this observation, Equation 8 defines an upper bound for $IS(o, o', t)$. By substituting Equation 8 into Equation 4, we estimate the upper of $IS(o, o', t)$ using Equation 9. If $IS(o, o', I, \epsilon)_{ub} < \theta$, we can safely prune (o, o') .

$$IS(o, o', t)_{ub} = \frac{\min(|\mathcal{F}(o, t)|, |\mathcal{F}(o', t)|)}{\max(|\mathcal{F}(o, t)|, |\mathcal{F}(o', t)|)} \geq IS(o, o', t) \quad (8)$$

$$IS(o, o', I, \epsilon)_{ub} = \frac{\sum_{i=1}^m IS(o, o', t_i^\epsilon)_{ub}}{m} \quad (9)$$

3.3.2 Algorithm Details. Algorithm 1 presents the pseudo code for the HBall-tree based Join Algorithm (HBJ-Alg). The algorithm takes as input two sets of moving objects, a root node of the HBall-tree, a repartitioning threshold, and an intersection similarity threshold. The output is denoted by $\mathcal{R}$ representing all intersection pairs. Initially, $\mathcal{R}$ is empty (Line 1). For each moving object $o \in \mathcal{A}$, we use a list $\mathcal{L}$ to store candidate nodes that may have intersection with $MR(o, I)$. Initially, $\mathcal{L}$ has one element N_r (Line 3). The HBall-tree index is constructed by inserting time-interval motion ranges of object set $\mathcal{B}$. While $\mathcal{L}$ is not empty, we call function $\mathcal{L}.pop()$ to pop a node from $\mathcal{L}$ (Line 5). Next, the HBall-tree based filtering occurs at the “circle level” to identify potential intersection candidates (Lines 6-9). Specifically, we retrieve all nodes within the Hball-tree such that $dist(o, N_c) \leq R + r_q$, indicating a possible intersection between $MR(o, I)$ and N_c . If N is not a leaf node, we examine its child nodes. For each child node N_c of N , we check the distance between o and N_c . If the distance is no greater than the sum of the radii of o and N_c , we push N_c into the priority queue (Lines 8–9). If N is a leaf node, we retrieve all entries in $N.I$. For every entry E , we calculate the Euclidean distance between $(E.x, E.y)$ and $(MR(o, I).x_c, MR(o, I).y_c)$, denoted by $dist(o, E)$. If $d(o, E) > E.r + o.r$, we can safely prune $(o, E.o)$ (Lines 12–13). Otherwise, we proceed with additional pruning steps. Specifically, we perform pruning enhancement to further prune incapable pairs (Lines 14–15). Only for these retained pairs, we conduct exact computation of the intersection similarity, in which we compute multiple time-point intersection similarities using Equation 2 (Lines 16–17). Finally, we return $\mathcal{R}$ as the results (Line 18).

Algorithm 1 HBJ($\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, \beta, \theta$)

Input: Two sets of moving objects, $\mathcal{A}$ and $\mathcal{B}$; A root node of the HBall-tree, N_r ; A time interval, I ; A relaxed parameter, ϵ ; A repartitioning threshold, β ; An intersection similarity threshold, θ ;
Output: The IS-Join results, $\mathcal{R}$;

```

1: Init:  $\mathcal{R} \leftarrow \emptyset$ ;
2: for  $o \in \mathcal{A}$  do
3:    $\mathcal{L} \leftarrow \{N_r\}$ ;
4:   while  $\mathcal{L} \neq \emptyset$  do
5:      $N \leftarrow \mathcal{L}.pop()$ ;
6:     if  $N$  is a non-leaf node then
7:       for  $N_c \in N.pt(child)$  do
8:         if  $dist(o, N_c) \leq N_c.R + o.r$  then
9:            $\mathcal{L}.push(N_c)$ ;
10:    else
11:      for  $E \in N.I$  do
12:        if  $dist(o, E) > E.r + o.r$  then
13:           $prune(o, E.o)$ ; ▷ pre-checking strategy
14:        if  $IS(o, E.o, I, \epsilon)_{ub} < \theta$  then
15:           $prune(o, E.o)$ ; ▷ pruning enhancement strategy
16:        if  $IS(o, E.o, I, \epsilon) \geq \theta$  then
17:           $\mathcal{R} \leftarrow \{\mathcal{R}, (o, E.o)\}$ ; ▷ refinement
18: return  $\mathcal{R}$ ;

```

3.3.3 Time Complexity. The construction of the HBall-tree requires $O(|\mathcal{B}| \times \log |\mathcal{B}|)$ time. The time complexity for the pre-checking is $O(|\mathcal{A}| \times \log |\mathcal{B}|)$. Let C' denote the candidate set. Refining the candidates has a time complexity of $O(|C'| \times |I|/\epsilon \times p)$. Thus, the total time complexity of the HBJ-Alg is $O((|\mathcal{A}| + |\mathcal{B}|) \times \log |\mathcal{B}| + |C'| \times |I|/\epsilon \times p)$. HBJ-Alg benefits from enhanced pruning strategies, which significantly reduce the number of candidates that require refinement.

4 Experimental Study

4.1 Experimental Setup

4.1.1 Data Preparation. The experimental study² is conducted on two real-world trajectory datasets. The first dataset, Geolife [44, 45], includes 17,621 trajectories from 182 users over four years and captures a range of outdoor movements, including daily commutes and recreational activities such as shopping and cycling. The second dataset, Porto [24], comprises over 1.7 million taxi trajectories recorded in Porto, Portugal over 19 months. The trajectory data for Porto is recorded at 15-second intervals, while Geolife exhibits a variable sampling frequency. Timestamps are set within a 24-hour range, disregarding the date, as most urban transportation movements occur daily. We calculate the maximum speed v_{max} of an object as the highest speed observed during a given time interval. Determining the true value or optimal setting of v_{max} is beyond the scope of this study. To ensure practical relevance, we exclude extreme trajectories with excessively large sampling intervals that exceeding an hour, as they offer limited utility in real-world applications, and thus are not considered in our study. We acknowledge that GPS signals in tunnel environments may be weak or entirely

²The code is open-source: <https://github.com/likemlike/IS-Join>

	Geolife	Porto
$ O $	$\{4,6,8,10,12\} \times 10^3$ 8K (default)	$\{1,2,3,4,5\} \times 10^4$ 10K (default)
$ I $	$\{10s, 20s, 30s, 40s, 50s\}$ 30s (default)	$\{15s, 30s, 45s, 60s, 75s\}$ 45s (default)
$ \mathcal{F} $	$\{2, 4, 6, 8, 10\} \times 10^5$ 600K (default)	$\{1, 2, 3, 4, 5\} \times 10^5$ 100K (default)
ϵ	$\{\frac{ I }{2}, \frac{ I }{4}, \frac{ I }{6}, \frac{ I }{8}, \frac{ I }{10}\}$ $\frac{ I }{6}$ (default)	$\{\frac{ I }{2}, \frac{ I }{4}, \frac{ I }{6}, \frac{ I }{8}, \frac{ I }{10}\}$ $\frac{ I }{6}$ (default)

Table 2: Evaluation parameter settings

lost, resulting in abnormal positions or low sampling rates. While explicitly modeling such outliers is beyond the scope of this study, our data preprocessing pipeline mitigates these effects by filtering out anomalous trajectories based on constraints such as longitude, latitude, trajectory length, and speed. After preprocessing, we retain 12,015 valid trajectories out of 18,670 in the Geolife dataset, and 1,392,385 out of 1,710,670 in the Porto dataset. Two groups of moving objects are randomly selected from the dataset with the same number of moving objects.

4.1.2 Compared Algorithms. We compare the performance of our proposal against a well-designed M-tree based join algorithm [43] (cf. Appendix), referred to as MBJ-Alg, which serves as our baseline. To assess the impact of the pruning enhancement strategy, we evaluate HBJ-Alg without pruning enhancement strategy, referred to as HBJ-Alg#. For efficiency evaluation, we measure both the pruning ratio and CPU time. The pruning ratio is calculated as $1 - |C|/|O|^2$, where C is the candidate set. The CPU time represents the average runtime of each round of the algorithms. We evaluate the performance of HBJ-Alg without the repartitioning strategy, denoted as HBJ-Alg*.

4.1.3 Parameter Settings. The parameter settings are listed in Table 2. The leaf node capacity is set to 10 for both the M*-tree and HBall-tree. By default, the repartition threshold α is 0.5. In our experiments, the feature set $\mathcal{F}$ consists of randomly generated POIs. Specifically, we first determine the minimum and maximum longitude and latitude values across all trajectories. Within these spatial boundaries, a fixed number of POIs are randomly generated using a uniform distribution. By default, consists of randomly generated POIs, with 100,000 POIs placed within the coordinate range of Porto and 600,000 POIs for Geolife. The Euclidean distance serves as the default distance metric. All the methods are implemented in Java and evaluated on Ubuntu equipped with an AMD Ryzen 5 CPU (3.6GHz) and 32GB memory, utilizing a single thread for execution. Unless stated otherwise, the experimental results are averaged over 20 independent trials, each with different trajectory and feature set inputs.

metric/dataset \ method	MBJ-Alg	HBJ-Alg
Candidate size (Geolife)	6540764	28862
Pruning ratio (Geolife)	89.7800%	99.9549%
Candidate size (Porto)	2142988	50072
Pruning ratio (Porto)	97.8570%	99.9499%

Table 3: Pruning performance

4.2 Performance Evaluation

4.2.1 Evaluation on Pruning Effectiveness. First, we examine the pruning effectiveness of the algorithms using default settings. The experimental results are shown in Table 3. Comparing the candidate size and pruning ratios of MBJ-Alg to those of the HBJ-Alg, we see that the candidate size of the HBJ-Alg is only 0.5% and 2.3% of that of MBJ-Alg for Geolife and Porto, respectively. These findings demonstrate that our proposal significantly prunes the search space.

We proceed to evaluate the performance of proposed algorithms under various parameter settings. Note that when we vary one parameter, the others are set to default values.

4.2.2 Effect of the Cardinality of Moving Objects. We investigate how varying object cardinalities $|O|$ impact algorithm performance. Intuitively, a larger $|O|$ causes more time-interval motion ranges to be processed, the runtime is expected to be higher for all algorithms. First, we evaluate the construction time of MBJ-Alg, HBJ-Alg, and HBJ-Alg*. The results are shown in Figure 6. It is clear that the MBJ-Alg requires more construction efforts than HBJ-Alg. Compared to HBJ-Alg*, HBJ-Alg shows similar performance with a slight improvement. This improvement is attributed to the repartitioning technique in HBJ-Alg, which, despite adding computation when repartitioning is triggered, reduces the total number of partitions and thereby potentially decreases construction time. Next, we analyze the total runtime of the proposed algorithms for IS-Join. As the number of moving objects increases, the computational workload grows due to the increased number of intersection candidates, leading to higher time complexity for all algorithms. From Figure 7, we observe a significant increase in CPU time for MBJ-Alg as the dataset size grows. For instance, with 12K moving objects in Geolife and 50K in Porto, MBJ-Alg's search time exceeds 3 seconds and 30 seconds, respectively, which is computationally expensive. In contrast, HBJ-Alg and HBJ-Alg# consistently outperform MBJ-Alg, demonstrating superior efficiency, particularly for large datasets. Notably, HBJ-Alg exhibits no significant degradation in CPU time, showcasing the scalability of our approach. Additionally, HBJ-Alg reduces runtime by 30% compared to HBJ-Alg#, highlighting the impact of the pruning enhancement strategy. Specifically, with 12,000 moving objects in Geolife and 50,000 in Porto, HBJ-Alg achieves a speedup of 3.1x for Geolife and 3.3x for Porto compared to MBJ-Alg.

4.2.3 Effect of the Number of Features. In our experiments, we specify the feature set of motion ranges as points of interest. Intuitively, a larger $|\mathcal{F}|$ within the same time-interval motion range suggests more potential intersection locations for two objects, which could lead to increased runtime due to additional location checks. As shown in Figure 8, a slight increase in runtime is observed across

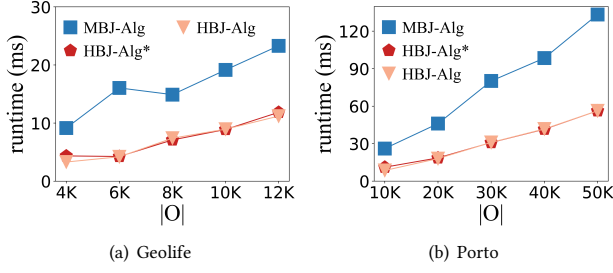
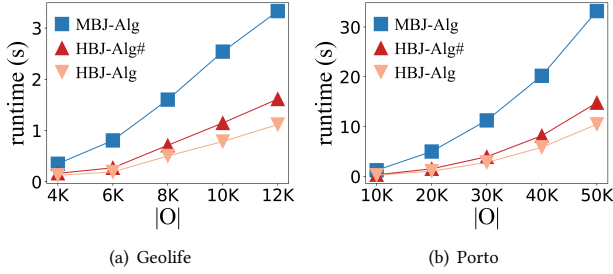
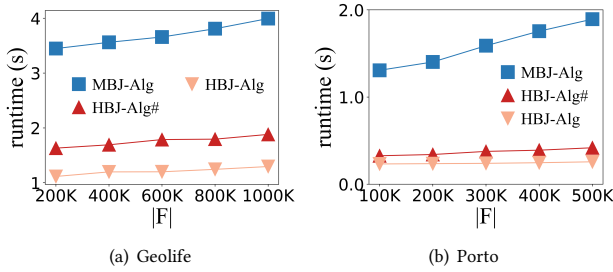
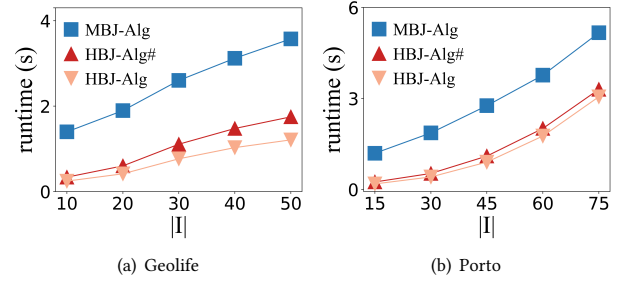
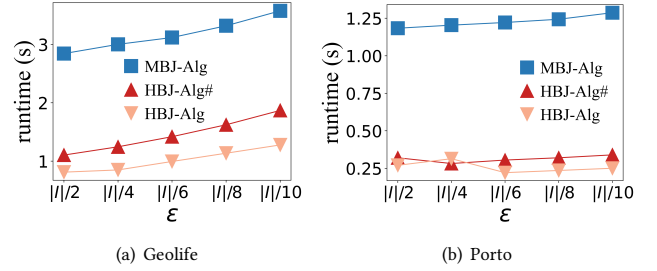
Figure 6: Effect of $|O|$ on the construction timeFigure 7: Effect of $|O|$ on the search time

Figure 8: Effect of the number of POI locations

all algorithms as $|F|$ grows. However, our pre-checking and pruning enhancement strategies effectively eliminate many object pairs early in the process, significantly reducing the number of candidates for POI intersection calculations. Consequently, the size of the POI set has only a minor impact on overall runtime, demonstrating the scalability of our approach.

4.2.4 Effect of the Time Interval. We use $|I|$ to denote the duration of a given time-interval. Specifically, $|I|$ varies from 10 to 50 seconds for Geolife and from 15 to 75 seconds for Porto. A larger $|I|$ increases the potential time-interval motion ranges, leading to more object pairs with intersections and a larger candidate size. Consequently, all algorithms require more computational effort to identify qualified pairs. Figure 9 illustrates the efficiency performance when we increase I . As expected, the runtime of all algorithms increases as we increase $|I|$, with HBJ-Alg consistently outperforming MBJ-Alg and HBJ-Alg# across all settings. By leveraging the M*-tree, we

Figure 9: Effect of $|I|$ Figure 10: Effect of the parameter ϵ

have improved filtering performance and reduced the number of candidates for further consideration. However, the M-tree's original design is not optimal for the IS-Join query due to time-consuming node splitting during construction. Moreover, indexing circles that cover only single time-interval motion ranges can lead to an expanded search space when dealing with large-scale moving objects, adversely affecting search performance.

4.2.5 Effect of ϵ . A smaller value of ϵ indicates more time-point motion ranges to be probed, bringing the relaxed similarity closer to the true intersection similarity, and thus increasing runtime, while leading to an expected increase in runtime. Figure 10 illustrates the efficiency performance when we vary the relaxing parameter ϵ . We vary the number of time-point motion ranges from 2 to 10. A slightly increasing trend is observed as ϵ varies. We can also observe that HBJ-Alg consistently demonstrates the best performance across all settings.

5 Related Work

5.1 Moving object indexing

Moving objects are typically represented as timestamped location samples [17, 45, 46]. R-trees [4, 14] and KD-trees [3, 40] are among the most popular indexing structures for efficient moving object queries. Each internal node in these trees corresponds to a rectangular region, with its subtree containing data points within that region. Additionally, M-trees [8, 9] and Ball-trees [22] are design to efficiently process queries within circular regions. Omohundro [26] examines trade-offs between construction time and tree quality by

comparing various Ball-tree construction methods. To enhance balance, Dolatshah *et al.* [12] modify space partitioning using principal component analysis to determine optimal splitting axes. Another related study [6] proposed a generalized contact search query over moving objects. Zhang *et al.* [43] study contact similarity queries and introduce the “UTM-tree” for indexing. However, no prior studies consider contact tracing based on motion ranges, as explored in our work.

5.2 Trajectory similarity join

Extensive research has explored trajectory similarity search and join [5, 15, 20, 21, 25, 37]. Among these studies, Shang *et al.* [31] use a linear combination method to integrate spatial and temporal similarities. To reduce system workload, velocity-based representation [27, 29, 33] has been proposed. Ding *et al.* [10] proposed a data-driven approach to formulate the problem as mining actual reachable region based on real historical trajectory dataset. For flexible trajectory similarity definitions, Bakalov *et al.* [2] represent trajectories as symbolic sequences using Piecewise Aggregate Approximation (PAA) [39], mapping PAA segments to symbols from a predefined alphabet. They construct a compact index to efficiently evaluate approximate joins, followed by a post-filtering step that refines query results with minimal trajectory data loading. Chen *et al.* [7] proposed a general framework for trajectory joins, including distance join and k nearest neighbor join. Additionally, Tampakis *et al.* [34] introduced a sub-trajectory join query that retrieves all “maximal” trajectory portions that are “similar”, identifying “maximally matching” sub-trajectories. Though these studies extend similarity concepts beyond mere proximity of location samples, to our knowledge, no prior work addresses intersection similarity join for motion ranges, making their solutions inapplicable to our problem.

6 Conclusion

In this paper, we introduce IS-Join, a novel motion range-aware similarity join framework for moving objects, which effectively captures the spatio-temporal interactions between objects by incorporating their potential movement regions rather than relying solely on discrete trajectory points. Unlike traditional trajectory similarity joins, IS-Join models the intersection similarity between motion ranges, enabling more flexible queries for applications such as traffic monitoring, wildlife tracking, and contact tracing. To efficiently process IS-Join queries at scale, we propose a HBall-tree indexing structure that integrates a repartitioning strategy. Additionally, we introduce pre-checking and pruning techniques to reduce unnecessary candidate evaluations, significantly improving efficiency. Our extensive experiments on two real-world datasets demonstrate that our proposal achieves up to 3x speedup compared to well-designed baselines.

Acknowledgments

This paper was supported by the National Key R&D Program of China 2024YFE0111800, and NSFC U22B2037, U21B2046, and 62032001.

References

- [1] Louai Alarabi, Saleh M. Basalamah, Abdeltawab M. Hendawi, and Mohammed Abdalla. 2021. TraceAll: A Real-Time Processing for Contact Tracing Using Indoor Trajectories. *Inf.* 12, 5 (2021), 202.
- [2] Petko Bakalov, Marios Hadjieleftheriou, Eamonn J. Keogh, and Vassilis J. Tsotras. 2005. Efficient trajectory joins using symbolic representations. In *MDM*, Panos K. Chrysanthos and George Samaras (Eds.). ACM, 86–93.
- [3] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 9 (1975), 509–517.
- [4] Thomas Brinkhoff, Hans-Peter Kriegel, and Bernhard Seeger. 1993. Efficient Processing of Spatial Joins Using R-Trees. In *SIGMOD*. 237–246.
- [5] Yanchuan Chang, Egemen Tanin, Gao Cong, Christian S. Jensen, and Jianzhong Qi. 2024. Trajectory similarity measurement: An efficiency perspective. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2293–2306.
- [6] Pingfu Chao, Dan He, Lei Li, Mengxuan Zhang, and Xiaofang Zhou. 2021. Efficient Trajectory Contact Query Processing. In *DASFAA*. 658–666.
- [7] Yun Chen and Jignesh M. Patel. 2009. Design and evaluation of trajectory join algorithms. In *17th ACM SIGSPATIAL International Symposium on Advances in Geographic Information Systems*. ACM, Washington, USA, 266–275.
- [8] Paolo Ciaccia, Marco Patella, Fausto Rabitti, and Pavel Zezula. 1997. Indexing Metric Spaces with M-Tree. In *SEBD*. 67–86.
- [9] Paolo Ciaccia, Marco Patella, and Pavel Zezula. 1997. M-tree: An Efficient Access Method for Similarity Search in Metric Spaces. In *VLDB*. 426–435.
- [10] Yichen Ding, Xun Zhou, Guojun Wu, Yanhua Li, Jie Bao, Yu Zheng, and Jun Luo. 2021. Mining Spatio-Temporal Reachable Regions With Multiple Sources over Massive Trajectory Data. *IEEE Trans. Knowl. Data Eng.* 33, 7 (2021), 2930–2942.
- [11] Zhongjun Ding, Ke Li, Lisi Chen, and Shuo Shang. 2025. Parallel Online Similarity Join over Trajectory Streams. In *Proceedings of the ACM on Web Conference 2025*. 3426–3437.
- [12] Mohamad Dolatshah, Ali Hadian, and Behrouz Minaei-Bidgoli. 2015. Ball*-tree: Efficient spatial indexing for constrained nearest-neighbor search in metric spaces. *CoRR* (2015).
- [13] SW Forrest, MR Recio, and PJ Seddon. 2022. Moving wildlife tracking forward under forested conditions with the SWIFT GPS algorithm. *Animal Biotelemetry* 10, 1 (2022), 19.
- [14] Antonin Guttman. 1984. R-Trees: A Dynamic Index Structure for Spatial Searching. In *SIGMOD*. 47–57.
- [15] Peng Han, Jin Wang, Di Yao, Shuo Shang, and Xiangliang Zhang. 2021. A Graph-based Approach for Trajectory Similarity Computation in Spatial Networks. In *KDD*. ACM, 556–564.
- [16] Xiangjie Kong, Qiao Chen, Mingliang Hou, Hui Wang, and Feng Xia. 2023. Mobility trajectory generation: a survey. *Artif. Intell. Rev.* 56, Supplement 3 (2023), 3057–3098.
- [17] Jiajia Li, Cancan Ni, Dan He, Lei Li, Xiufeng Xia, and Xiaofang Zhou. 2023. Efficient kNN query for moving objects on time-dependent road networks. *VLDB J.* 32, 3 (2023), 575–594.
- [18] Ke Li, Lisi Chen, and Shuo Shang. 2020. Towards Alleviating Traffic Congestion: Optimal Route Planning for Massive-Scale Trips. In *IJCAI*. 3400–3406.
- [19] Ke Li, Lisi Chen, Shuo Shang, Haiyan Wang, Yang Liu, Panos Kalnis, and Bin Yao. 2022. Towards Controlling the Transmission of Diseases: Continuous Exposure Discovery over Massive-Scale Moving Objects. In *IJCAI*. 3891–3897.
- [20] Ke Li, Hongyu Wang, Ziwen Chen, and Lisi Chen. 2023. Relaxed group pattern detection over massive-scale trajectories. *Future Generation Computer Systems* 144 (2023), 131–139.
- [21] Anqi Liang, Bin Yao, Bo Wang, Yinpei Liu, Zhida Chen, Jiong Xie, and Feifei Li. 2024. Sub-trajectory clustering with deep reinforcement learning. *VLDB J.* 33, 3 (2024), 685–702.
- [22] Ting Liu, Andrew W. Moore, and Alexander G. Gray. 2006. New Algorithms for Efficient High-Dimensional Nonparametric Classification. *J. Mach. Learn. Res.* 7 (2006), 1135–1158.
- [23] Andrew W. Moore. 2000. The Anchors Hierarchy: Using the Triangle Inequality to Survive High Dimensional Data. In *UAI*. 397–405.
- [24] Luis Moreira-Matias, João Gama, Michel Ferreira, João Mendes-Moreira, and Luís Damas. 2013. Predicting Taxi-Passenger Demand Using Streaming Data. *IEEE Trans. Intell. Transp. Syst.* 14, 3 (2013), 1393–1402.
- [25] Sarana Nutanong, Rui Zhang, Egemen Tanin, and Lars Kulik. 2010. Analysis and evaluation of V*-kNN: an efficient algorithm for moving kNN queries. *VLDB J.* 19, 3 (2010), 307–332.
- [26] Stephen M Omohundro. 1989. *Five balltree construction algorithms*. International Computer Science Institute Berkeley.
- [27] Jignesh M. Patel, Yun Chen, and V. Prasad Chakka. 2004. STRIPES: An Efficient Index for Predicted Trajectories. In *SIGMOD*. 637–646.
- [28] Dieter Pfoser and Christian S. Jensen. 1999. Capturing the Uncertainty of Moving-Object Representations. In *SSD*. 111–132.
- [29] Simonas Saltenis, Christian S. Jensen, Scott T. Leutenegger, and Mario Alberto López. 2000. Indexing the Positions of Continuously Moving Objects. In *SIGMOD*. 331–342.

- [30] Hanan Samet. 1984. The Quadtree and Related Hierarchical Data Structures. *ACM Comput. Surv.* 16, 2 (1984), 187–260.
- [31] Shuo Shang, Lisi Chen, Zhewei Wei, Christian S. Jensen, Kai Zheng, and Panos Kalnis. 2018. Parallel trajectory similarity joins in spatial networks. *VLDB J.* 27, 3 (2018), 395–420.
- [32] Shuo Shang, Lisi Chen, Kai Zheng, Christian S. Jensen, Zhewei Wei, and Panos Kalnis. 2019. Parallel Trajectory-to-Location Join. *TKDE* 31, 6 (2019), 1194–1207.
- [33] A. Prasad Sistla, Ouri Wolfson, Sam Chamberlain, and Son Dao. 1997. Modeling and Querying Moving Objects. In *ICDE*. 422–432.
- [34] Panagiotis Tampakis, Christos Doukeridis, Nikos Pelekis, and Yannis Theodoridis. 2020. Distributed Subtrajectory Join on Massive Datasets. *ACM Trans. Spatial Algorithms Syst.* 6, 2 (2020), 8:1–8:29.
- [35] Goce Trajcevski, Alok Choudhary, Ouri Wolfson, Li Ye, and Gang Li. 2010. Uncertain range queries for necklaces. In *MDM*. 199–208.
- [36] Chalavadi Vishnu, Vineel Abhinav, Debadiya Roy, C. Krishna Mohan, and Ch. Sobhan Babu. 2023. Improving Multi-Agent Trajectory Prediction Using Traffic States on Interactive Driving Scenarios. *IEEE Robotics Autom. Lett.* 8, 5 (2023), 2708–2715.
- [37] Phillip G. D. Ward, Zhen He, Rui Zhang, and Jianzhong Qi. 2014. Real-time continuous intersection joins over large sets of moving objects using graphic processing units. *VLDB J.* 23, 6 (2014), 965–985.
- [38] Jianqiu Xu, Hua Lu, and Zhifeng Bao. 2020. IMO: A Toolbox for Simulating and Querying "Infected" Moving Objects. *Proc. VLDB Endow.* 13, 12 (2020), 2825–2828.
- [39] Byoung-Kee Yi and Christos Faloutsos. 2000. Fast Time Sequence Indexing for Arbitrary Lp Norms. In *VLDB*. Morgan Kaufmann, 385–394.
- [40] Meihui Zhang, Su Chen, Christian S. Jensen, Beng Chin Ooi, and Zhenjie Zhang. 2009. Effectively Indexing Uncertain Moving Objects for Predictive Queries. *Proc. VLDB Endow.* 2, 1 (2009), 1198–1209.
- [41] Rui Zhang, Dan Lin, Kotagiri Ramamohanarao, and Elisa Bertino. 2008. Continuous Intersection Joins Over Moving Objects. In *ICDE*. 863–872.
- [42] Rui Zhang, Jianzhong Qi, Dan Lin, Wei Wang, and Raymond Chi-Wing Wong. 2012. A highly optimized algorithm for continuous intersection join queries over moving objects. *VLDB J.* 21, 4 (2012), 561–586.
- [43] Xichen Zhang, Suprio Ray, Farzaneh Shoeleh, and Rongxing Lu. 2021. Efficient Contact Similarity Query over Uncertain Trajectories. In *EDBT*. 403–408.
- [44] Yu Zheng, Quannan Li, Yukun Chen, Xing Xie, and Wei-Ying Ma. 2008. Understanding mobility based on GPS data. In *UbiComp*. 312–321.
- [45] Yu Zheng, Xing Xie, and Wei-Ying Ma. 2010. GeoLife: A Collaborative Social Networking Service among User, Location and Trajectory. *IEEE Data Eng. Bull.* 33, 2 (2010), 32–39.
- [46] Yu Zheng, Lizhu Zhang, Xing Xie, and Wei-Ying Ma. 2009. Mining interesting locations and travel sequences from GPS trajectories. In *WWW*. 791–800.

A M*-tree based Join Algorithm

First, we introduce a M*-tree based Join algorithm (MBJ-Alg) for computing the IS-Join, which serves as a baseLine method. The M*-tree is an extension of the M-tree [8, 9]. The MBJ-Alg is a two-phase algorithm incorporating a pre-checking strategy, where an M-tree variant is used to enhance candidate filtering. In Phase 1, the M*-tree uses approximate circles to index time-interval motion ranges. This circular indexing allows for early pruning of unqualified objects, concentrating on candidates more likely to intersect. In Phase 2, candidates are evaluated at the time-point motion range level. We proceed to present the structure of the M*-tree, and its application in IS-Join queries.

A.1 The M*-tree

An illustration of the M*-tree structure is shown in Figure 11, with a specified capacity of $M = 4$, indicating the maximum number of entries per leaf node. Approximate circles are treated as individual entries and stored independently in the leaf nodes. We use approximate circles to represent time-interval motion ranges and index them within the M*-tree. Each circle is stored as an entry $E = \{o, x, y, r, MR(o, I)\}$, where o is the object identifier, (x, y) represents the center of the circle, r is the radius, and $MR(o, I)$ contains the time-interval motion range. Each node N in the M*-tree selects an entry E in the leaf nodes as its routing object, which is formatted

as $N = \{E, R, pt(child)\}$. Here, E is the routing object, R is a radius that can cover all entries (i.e., circles) stored in N , and $pt(child)$ is a pointer that points to its child nodes. Note that the M*-tree follows established practices from previous studies [9, 43] in terms of the entry insertion policy, splitting strategies, and selection strategies for routing objects, adapting these strategies to the M*-tree structure.

It is crucial to note that time-interval motion ranges within the same M*-tree are temporally aligned. We index all motion ranges for a specific time interval, and when the time interval changes, the M*-tree index is reconstructed.

A.2 Intersection Checking

We begin with a query entry $E_q = (q, x_q, y_q, r_q, MR(q, I))$ and the root node of the M*-tree. The goal is to retrieve all entries $E = \{o, x, y, r, MR(o, I)\}$ that may intersect with E_q . To determine potential intersections, we calculate the distance $d(q, E)$ between the query object q and each entry E by $\sqrt{(x_q - x)^2 + (y_q - y)^2}$. If $d(q, E) \leq r + r_q$, then the time-interval motion ranges $MR(q, I)$ and $MR(o, I)$ may intersect. Here, r and r_q represent the radii of the entries E and E_q , respectively. For each query object q , we index entries E such that $d(q, E) \leq r + r_q$, and update candidates by adding (q, o) , where o is the object identifier associated with the entry E .

A.3 Algorithm Details

Algorithm 2 MBJ($\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, \theta$)

Input: Two sets of moving objects, $\mathcal{A}$ and $\mathcal{B}$;

A root node of M*-tree, N_r ;

A time interval, I ;

A partition parameter, ϵ ;

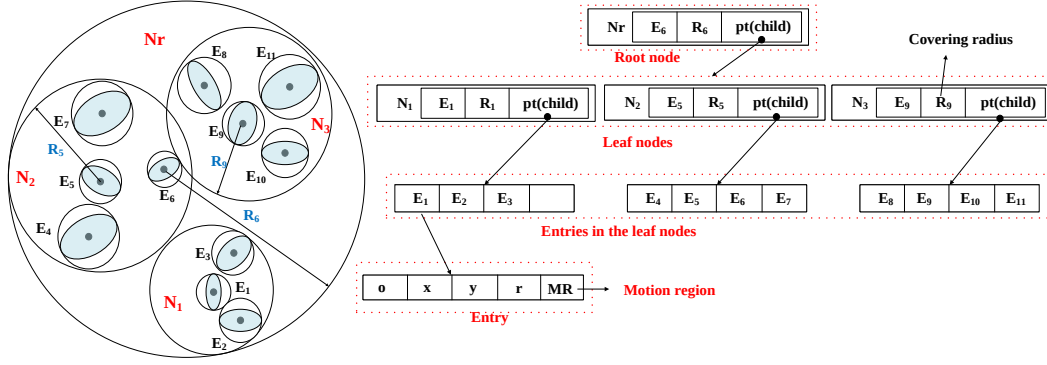
An intersection similarity threshold, θ ;

Output: $\mathcal{R} = \{(o, o') | IS(o, o', I, \epsilon) \geq \theta, (o, o') \in \mathcal{A} \times \mathcal{B}\}$

```

1: Init:  $\mathcal{R} \leftarrow \emptyset$ .
2: for  $o \in \mathcal{A}$  do
3:    $\mathcal{L} \leftarrow \{N_r\}$ ;
4:   while  $\mathcal{L} \neq \emptyset$  do
5:      $N \leftarrow \mathcal{L}.pop()$ ;
6:     if  $N$  is a non-leaf node then
7:       for  $N_c \in N.pt(child)$  do
8:         if  $d(o, N_c) \leq N_c.R + o.r$  then
9:            $\mathcal{L}.push(N_c)$ ;
10:    else
11:      if  $d(o, N) > N.R + o.r$  then
12:         $prune(o, N.o)$ ; ▷ pre-checking strategy
13:      if  $IS(o, N.o, I, \epsilon) \geq \theta$  then
14:         $\mathcal{R} \leftarrow \{\mathcal{R}, (o, N.o)\}$ ; ▷ refinement
15: return  $\mathcal{R}$ ;
```

Algorithm 2 presents the pseudo code of the M*-tree based two-phase pruning matching Algorithm (MBJ-Alg). The input consists of two sets of moving objects, a root node of M*-tree, a time interval with a partition parameter ϵ , and a similarity threshold θ . We use a list $\mathcal{R}$ to store all satisfied object pairs. Initially, $\mathcal{R}$ is empty (Line

Figure 11: The structure of the M^* -tree

1). The algorithm comprises two phases: (1) M^* -tree based pre-checking (Lines 3–12), and (2) refinement process (Lines 13–14). In the pre-checking phase, we get a set of candidates based on the M^* -tree. We use a list $\mathcal{L}$ to store nodes with objects that may intersect moving objects, and $\mathcal{L}$ initially contains only one element, namely the root node (Line 3). While $\mathcal{L}$ is not empty, we retrieve a M^* -tree node N with the minimal $d(o, N)$ (Line 5). If N is not a leaf node, we consider its child nodes. For each child node N_c of N , we check

the distance between o and N_c . If the distance is no larger than the sum of the radii of o and N_c , we push N_c into the priority queue (Lines 7–9). If N is a leaf node and $d(o, N) > N.R + o.r$, we can safely prune $(o, N.o)$ (Lines 11–12). If the intersection similarity is no less than the similarity threshold θ , we update the result set $\mathcal{R}$ by adding $(o, N.o)$ (Line 14). Finally, we return $\mathcal{R}$ as the resulting intersection pairs (Line 15).